

C1.8 The Data Watchpoint and Trace unit

The *Data Watchpoint and Trace* (DWT) unit provides the following:

- Comparators, that support:
 - Watchpoints, causes the processor to enter Debug state or take a DebugMonitor exception.
 - Data tracing.
 - Signaling for use with an external resource, for example an ETM.
 - PC value tracing.
 - Cycle count matching.
- Additional PC sampling:
 - PC sample trace output as a result of a cycle count event.
 - External PC sampling using a PC sample register.
- Exception trace.
- Performance profiling counters.

Which DWT features are supported is IMPLEMENTATION DEFINED. See [Debug support in Armv7-M on page C1-683](#) for information on how to determine the level of DWT support in an implementation.

Many DWT operations generate DWT trace data, using the packet protocol described in [Appendix D4 Debug ITM and DWT Packet Protocol](#). The DWT forwards these packets to the ITM for transmission, see [How the ITM relates to other debug components on page C1-713](#). Software must set the ITM_TCR.TXENA bit to 1 to enable transmission of these DWT packets, see [Trace Control Register, ITM_TCR on page C1-716](#).

———— Note ————

Transmission of packets generated by the ITM or the DWT unit requires the processor to implement and enable the cycle counter, indicated by DWT_CTRL.NOCYCCNT being RAZ, and DWT_CTRL.CYCCNTENA being set to 1, see [Control register, DWT_CTRL on page C1-737](#).

The following sections describe the operation of the different features of the DWT unit:

- [The DWT comparators.](#)
- [Exception trace support on page C1-731.](#)
- [CYCCNT cycle counter and related timers on page C1-731.](#)
- [Profiling counter support on page C1-734.](#)
- [Exception trace support on page C1-731.](#)

[DWT register summary on page C1-736](#) summarizes the DWT unit registers, and the remaining subsections in this section describe each of the DWT registers.

C1.8.1 The DWT comparators

The DWT unit can include between 0 and 15 comparators. The DWT_CTRL.NUMCOMP field indicates the number of implemented comparators, see [Control register, DWT_CTRL on page C1-737](#).

When a DWT implementation includes one or more comparators, which comparator features are supported is IMPLEMENTATION DEFINED. [Checking the implemented features of the DWT comparators on page C1-730](#) describes how software can find which features are supported. The remainder of this summary describes an implementation that includes all features.

A DWT comparator compares one of the following with the value held in its DWT_COMP register:

- A data address.
- An instruction address.
- A data value.
- The cycle count value, for comparator 0 only.

For address matching, the comparator can use a mask, so it matches a range of addresses.

On a successful match, the comparator generates one of the following:

- One or more DWT Data trace packets, containing one or more of:
 - The address of the instruction that caused a data access.
 - Bits[15:0] of the data access address.
 - The matched data value.
- A watchpoint debug event, on either the PC value or the accessed data address.
- A **CMPMATCH[N]** event, that signals the match outside the DWT unit.

A watchpoint debug event either generates a DebugMonitor exception, or causes the processor to halt execution and enter Debug state.

For each implemented comparator, a set of registers define the comparator operation. For comparator *n*:

- DWT_COMP*n* holds a value for comparison, see [Comparator registers, DWT_COMP*n* on page C1-745](#).
- For address comparisons, DWT_MASK*n* holds a mask for use when comparing with the value of DWT_COMP*n*, see [Comparator Mask registers, DWT_MASK*n* on page C1-745](#). Using a mask means the comparator matches on a range of addresses, defined by the unmasked most significant bits of the address.
- DWT_FUNCTION*n* defines the operation of the comparator, see [Comparator Function registers, DWT_FUNCTION*n* on page C1-746](#). This definition includes:
 - Whether the comparison is an access address comparison, a data value comparison, or, for comparator 0 only, a cycle count comparison.
 - For a data value comparison, the size of the data unit to compare, byte, halfword, or word.
 - Whether this comparator is linked with another comparator, to define a more complex comparison, for example a comparison of both the instruction address and the associated data value.
 - For implemented comparators up to and including comparator 3, whether a successful address comparison generates a DWT Data trace packet. See [Data trace packets discriminator IDs 8-23 on page D4-793](#) for more information about these packets.
 - The comparator function. That is, the type of comparison to make and the action to take if the comparison succeeds. For more information see [Summary of DWT comparator functions](#).

When multiple comparator matches occur on execution of an instruction, the DWT always generates any required watchpoint event or **CMPMATCH[N]** event. However it is UNPREDICTABLE whether it generates Data trace packets.

If a comparator match requires the DWT to generate a Data trace packet, but it cannot do so because the DWT output buffer is full, the ITM generates an Overflow packet, see [Overflow packet on page D4-783](#).

Summary of DWT comparator functions

The DWT_FUNCTION register defines the required operation of the comparator. The following fields together define the required function:

- DATAVMATCH, bit[8].
- CYCMATCH, bit[7]. On all comparators except comparator 0, the CYCMATCH bit is UNK/SBZP.
- FUNCTION, bits[3:0].

For some functions, other fields in the DWT_FUNCTION register give more information about the required operation. shows how the DATAVMATCH and CYCMATCH fields define the required comparison type, and the use of the corresponding DWT_MASK and DWT_COMP registers, and references subsections that describe how the FUNCTION field defines the supported comparisons of that type.

Table C1-13 DWT_FUNCTION register comparison type definition

DWT_FUNCTION bits:		Comparison type and DWT_COMP use	DWT_MASK use	Functions
DATAVMATCH	CYCMATCH			
0	0	Address	Mask value	See Address comparison functions
0	1 ^a	Cycle count	SBZ	See Cycle count comparison functions on page C1-724
1 ^b	0	Data value	SBZ	See Data value comparison functions on page C1-725
1 ^b	1 ^a	-	-	Operation is UNPREDICTABLE

- a. On comparator 0 only, and only if the implementation supports cycle count comparison, otherwise, DWT_FUNCTION.CYCMATCH is UNK/SBZP.
- b. Only if the comparator supports data value matching, otherwise, DWT_FUNCTION.DATVMATCH is RAZ/WI.

Address comparison functions

Software selects these functions by setting DWT_FUNCTION_n.DATAVMATCH to 0 and, for comparator 0, DWT_FUNCTION0.CYCMATCH to 0.

For these functions:

- The DWT_FUNCTION_n.FUNCTION field defines the required access types to be compared:
 - Data accesses, or instruction accesses.
 - Read accesses, write accesses, or read and write accesses.
 It also defines the action performed on a successful match.
- For some functions, the DWT_FUNCTION_n.EMITRANGE bit affects the action performed on a successful match.
- The comparator compares the address of each memory access of the type specified by the FUNCTION field with the value held in the corresponding DWT_COMP register. If the values match it performs the specified operation.
- Software can program a comparator to match on a range of addresses by:
 - Programming DWT_COMP with the most significant address bits required for the match, with the least significant bits programmed as zeros.
 - Programming DWT_MASK with a corresponding mask size, to be applied to the access address when making the comparison.

For example, to match on a 16-byte block of addresses software must:

- Program DWT_COMP bits[31:4] with the value of the most significant bits of the required addresses, and bits[3:0] as 0b0000.
- Program DWT_MASK with a value of four, to mask address bits[3:0] from the comparison.
- The DWT_FUNCTION_n.DATAVADDR1, DATAVADDR0, and DATAVSIZE fields are all SBZ. Operation is UNPREDICTABLE if software programs any of these fields to a non-zero value.

It is IMPLEMENTATION DEFINED whether a vector table read performed as part of exception processing is considered as a memory read-access for the purpose of watchpoint matching.

[Table C1-14 on page C1-722](#) shows the supported address comparison functions. In this table, in the *Compared accesses* column:

- Daddr indicates that the comparator compares the address for data accesses.

- Iaddr indicates that the comparator compares the address for instruction fetches.
- RO indicates that the comparator compares the address only for read accesses.
- WO indicates that the comparator compares the address only for write accesses.
- RW indicates that the comparator compares the address for read and write accesses.

Table C1-14 DWT address comparison functions

DWT_FUNCTION n fields		Compared accesses		Action on successful match
FUNCTION	EMITRANGE			
0000	x	-	-	Comparator disabled, or part of a LinkAddr ^a comparison
0001	0	Daddr	RW	Generate Data trace PC value packet ^b
	1	Daddr	RW	Generate Data trace address packet ^b
0010	0	Daddr	RW	Generate Data trace data value packet ^b
	1	Daddr	RW	Generate Data trace address and data value packets ^b
0011	0	Daddr	RW	Generate Data trace PC value and data value packets ^b
	1	Daddr	RW	Generate Data trace address and data value packets ^b
0100	x	Iaddr	-	Generate PC watchpoint debug event ^c
0101	x	Daddr	RO	Generate watchpoint debug event ^c
0110	x	Daddr	WO	Generate watchpoint debug event ^c
0111	x	Daddr	RW	Generate watchpoint debug event ^c
1000	x	Iaddr	-	Generate CMPMATCH[N] event ^d
1001	x	Daddr	RO	Generate CMPMATCH[N] event ^d
1010	x	Daddr	WO	Generate CMPMATCH[N] event ^d
1011	x	Daddr	RW	Generate CMPMATCH[N] event ^d
1100	0	Daddr	RO	Generate Data trace data value packet ^b
	1	Daddr	RO	Generate Data trace address packet ^b
1101	0	Daddr	WO	Generate Data trace data value packet ^b
	1	Daddr	WO	Generate Data trace address packet ^b
1110	0	Daddr	RO	Generate Data trace PC value and data value packets ^b
	1	Daddr	RO	Generate Data trace address and data value packets ^b
1111	0	Daddr	WO	Generate Data trace PC value and data value packets ^b
	1	Daddr	WO	Generate Data trace address and data value packets ^b

a. For more information see [LinkAddr support on page C1-728](#). In this case, this comparator compares the data access address, and the linked data value comparator defines the action taken when both comparators match.

b. For more information, see [Data trace packet generation on page C1-729](#).

c. For more information, see [Watchpoint debug event generation on page C1-729](#).

d. For more information, see [CMPMATCH\[N\] event generation on page C1-730](#).

Comparator behavior for instruction address matching

All comparators support instruction address matching. Comparator operation is UNPREDICTABLE unless:

- If masking is not used, the DWT_COMP n value is halfword-aligned.
- If masking is used, the DWT_COMP n value is the masked address for comparison.

If software sets a match on the address of a NOP or IT instruction, whether the event occurs or not is UNPREDICTABLE.

The following pseudocode shows the comparator behavior for instruction address matching:

```
// InstructionAddressMatch()
// =====

boolean InstructionAddressMatch(integer N, bits(32) Iaddr)

    assert N < UInt(DWT_CTRL.NUMCOMP);

    valid = ((DWT_FUNCTION[N].FUNCTION == '0100' || DWT_FUNCTION[N].FUNCTION == '1000') &&
             DWT_FUNCTION[N].DATAVMATCH == '0' &&
             (N != 0 || DWT_FUNCTION[N].CYCMATCH == '0'));

    if valid then
        mask = ZeroExtend(Ones(UInt(DWT_MASK[N].MASK)), 32);
        // UNPREDICTABLE if COMP does not meet alignment and masking conditions
        if !IsZero(DWT_COMP[N] AND mask) || !IsZero(DWT_COMP[N]<0>) then UNPREDICTABLE;
        match = ((Iaddr AND NOT(mask)) == DWT_COMP[N]);
    else
        match = FALSE;

    return match;
```

Comparator behavior for data address matching

A data address comparison can be linked to a data value comparison, see [LinkAddr support on page C1-728](#). If so, a match occurs only if both the data address comparison and the data value comparison succeed, and this subsection defines only the behavior of the data address comparison.

For a data address comparison, the implementation must address test all memory accesses of the specified type, RO, WO, or RW, for which the range of watched addresses lies between the start address of the transaction and the next word aligned address. It is IMPLEMENTATION DEFINED whether the comparison matches some or all unaligned memory accesses that access a watched location across a word boundary.

The following pseudocode shows the comparator behavior for data address matching:

```
// DataAddressMatch()
// =====

boolean DataAddressMatch(integer N, bits(32) Daddr, integer size, boolean read,
                        boolean is_linked)

    assert N < UInt(DWT_CTRL.NUMCOMP);
    assert size == 1 || size == 2 || size == 4;

    if is_linked then
        if DWT_FUNCTION[N].FUNCTION != '0000' then UNPREDICTABLE;
        valid = TRUE;
    elseif DWT_FUNCTION[N].FUNCTION == '0000' || DWT_FUNCTION[N].DATAVMATCH == '1' ||
        (N == 0 && DWT_FUNCTION[N].CYCMATCH == '1') then
        valid = FALSE;
    else
        case DWT_FUNCTION[N].FUNCTION of
            when '0100', '1000' // See InstructionAddressMatch()
                valid = FALSE;
            when '0101', '1001', '11x0' // Reads
                valid = read;
            when '0110', '1010', '11x1' // Writes
                valid = !read;
```

```

        otherwise // Reads and writes
            valid = TRUE;

    if valid then
        // UNPREDICTABLE if the base compare address is not properly aligned
        mask = ZeroExtend(Ones(UInt(DWT_MASK[N].MASK)), 32);
        if !IsZero(DWT_COMP[N] AND mask) then UNPREDICTABLE;

        // compute start and end addresses of compared region
        comp_start = UInt(DWT_COMP[N]);
        comp_end = comp_start + UInt(mask);

        // compute start and end addresses of access
        access_start = UInt(Daddr);
        access_end = UInt(Daddr) + size - 1;

        // Implementations can terminate matching at the word address boundary
        if IMPLEMENTATION_DEFINED "condition" then
            if access_end<31:2> != access_start<31:2> then
                access_end = UInt(access_start<31:2>:'11');

        match = ((access_start >= comp_start && access_start <= comp_end) ||
            (access_end >= comp_start && access_end <= comp_end) ||
            (access_start <= comp_start && access_end >= comp_end));
    else
        match = FALSE;

    return match;

```

Cycle count comparison functions

Comparator 0 supports cycle count comparisons only if the DWT_CTRL.NOCYCCNT bit is RAZ, see [Control register, DWT_CTRL on page C1-737](#). Other comparators never support cycle count comparisons.

When comparator 0 supports cycle count comparisons, software selects these functions by setting DWT_FUNCTION0.DATAVMATCH to 0 and DWT_FUNCTION0.CYCMATCH to 1.

For these functions:

- The DWT_FUNCTION0.FUNCTION field defines action performed on a successful match.
- For each instruction committed for execution, the comparator compares the current cycle count value with the value in the DWT_COMP0 register. If the values match it performs the required function.
- The DWT_FUNCTION0.DATAVADDR1, DATAVADDR0, DATAVSIZE, and EMITRANGE fields are all SBZ. Operation is UNPREDICTABLE if software programs any of these fields to a non-zero value and a match occurs.
- The DWT_MASK register is SBZ. Operation is UNPREDICTABLE if software programs this register to a non-zero value and a match occurs.

[Table C1-15](#) shows the cycle count comparison functions.

When DWT_FUNCTION0.DATAVMATCH is set to 0 and DWT_FUNCTION0.CYCMATCH is set to 1, software must program the FUNCTION field to a value with an effect, as shown in [Table C1-15](#), that is not UNPREDICTABLE.

Table C1-15 DWT cycle count comparison functions

DWT_FUNCTION0.FUNCTION	Action on successful match
0000	Comparator disabled
0001	Generate Data trace PC value packet ^a
001x	UNPREDICTABLE
0100	Generate watchpoint debug event ^b

Table C1-15 DWT cycle count comparison functions (continued)

DWT_FUNCTION0.FUNCTION	Action on successful match
0101	UNPREDICTABLE
011x	UNPREDICTABLE
1000	Generate CMPMATCH[N] event ^c
1001	UNPREDICTABLE
101x	UNPREDICTABLE
11xx	UNPREDICTABLE

- a. For more information, see [Data trace packet generation on page C1-729](#).
b. For more information, see [Watchpoint debug event generation on page C1-729](#).
c. For more information, see [CMPMATCH\[N\] event generation on page C1-730](#).

Comparator behavior for cycle count matching

The following pseudocode shows the comparator behavior for cycle count matching:

———— Note ————

The DWT_CTRL.NOCYCCNT bit field is RAO when the implementation does not support cycle counter comparison. In this case validCYCMATCH is always FALSE.

```
// CycCountMatch
// =====

boolean CycCountMatch(integer N)

    assert N < UInt(DWT_CTRL.NUMCOMP) && DWT_CTRL.NOCYCCNT == '0';

    valid = N == 0 && DWT_FUNCTION[N].CYCMATCH == '1' && DWT_FUNCTION[0].FUNCTION != '0000';

    if valid then
        if DWT_FUNCTION[N].DATAVMATCH == '1' then UNPREDICTABLE;
        if DWT_FUNCTION[N].FUNCTION IN {"001x", "0101", "011x", "1001", "101x", "11xx"} then
            UNPREDICTABLE;
        match = (CYCCNT == DWT_COMP[N]);
    else
        match = FALSE;

    return match;
```

Data value comparison functions

Software selects these functions by setting DWT_FUNCTION_n.DATAVMATCH to 1 and, for comparator 0, DWT_FUNCTION0.CYCMATCH to 0. Not all comparators support data value comparisons. If a comparator does not support data value comparison, DWT_FUNCTION_n.DATAVMATCH is RAZ/WI.

The registers that support data value comparisons, if any, must be sequentially-numbered comparators starting at comparator 1. Comparator 0 must not support data value comparisons unless all other comparators support data value comparisons.

For these functions:

- The DWT_FUNCTION_n.FUNCTION field defines whether the comparison is made for read accesses, write accesses, or read and write accesses. It also defines the action performed on a successful match.
- The DWT_FUNCTION_n.DATAVSIZE field specifies the size of the required data comparison, word, halfword, or byte:
 - Software must program DWT_COMP_n with the required data match value, in little-endian order.

- For halfword comparisons software must program the required match value into both halfwords of DWT_COMP n .
- For byte comparisons software must program the required match value into each byte of DWT_COMP n .
- The comparator compares the value memory accesses of the type specified by the FUNCTION field with the value held in the corresponding DWT_COMP register. If the values match it performs the specified operation.
Software can use the LinkAddr feature to restrict data value comparison to particular memory addresses, see [LinkAddr support on page C1-728](#). Otherwise, the comparator tests the data value for all data accesses of the specified type.
[LinkAddr support on page C1-728](#) also describes how DWT_FUNCTION n .LNK1ENA indicates the programming options for the DATAVADDR0 and DATAVADDR1 fields.
- The DWT_FUNCTION n .EMITRANGE bit is SBZ. Operation is UNPREDICTABLE if software programs this bit to 1 and a match occurs.
- The DWT_MASK n register is SBZ. Operation is UNPREDICTABLE if software programs this register to a non-zero value and a match occurs.

Table C1-16 shows the supported data value comparison functions. In the *Compared accesses* column of this table:

- RO indicates that the comparator compares the address only for read accesses.
- WO indicates that the comparator compares the address only for write accesses.
- RW indicates that the comparator compares the address for read and write accesses.

Table C1-16 DWT data value comparison functions

DWT_FUNCTION n .FUNCTION	Compared accesses	Action on successful match
0000	-	Disabled
00x1	-	UNPREDICTABLE
001x	-	UNPREDICTABLE
0100	-	UNPREDICTABLE
0101	RO	Generate watchpoint debug event ^a
0110	WO	Generate watchpoint debug event ^a
0111	RW	Generate watchpoint debug event ^a
1000	-	UNPREDICTABLE
1001	RO	Generate CMPMATCH[N] event ^b
1010	WO	Generate CMPMATCH[N] event ^b
1011	RW	Generate CMPMATCH[N] event ^b
11xx	-	UNPREDICTABLE

a. For more information, see [Watchpoint debug event generation on page C1-729](#).

b. For more information, see [CMPMATCH\[N\] event generation on page C1-730](#).

Comparator behavior for data value matching

Data value matching generates a match where the data value in the DWT_COMP n register is the same as the data access value, or is a partial match if the size of the data access is larger than that specified in the DWT_FUNCTION n .DATAVSIZE field.

A data value comparison can be linked to an address comparison, see [LinkAddr support on page C1-728](#). If it is, whether the data value matching is exact on the data address is IMPLEMENTATION DEFINED. If matching is exact, a match is generated only if the data value in DWT_COMP n precisely matches the value in memory at the address specified in the linked address comparator DWT_COMP m . In an implementation that permits inexact matches, the conditions that generate an inexact match are IMPLEMENTATION DEFINED. [Example C1-1](#) shows the difference between exact and inexact matching.

Example C1-1 Exact and inexact dependence on a linked address comparison

If DWT_FUNCTION n .DATAVSZ specifies a halfword comparison, and linked address comparator DWT_COMP m defines a halfword-aligned address with DWT_MASK m specifying a 1-bit address mask, then:

- An exact match matches only an access where the halfword at address DWT_COMP m is accessed with the value DWT_COMP n .
- An inexact match can match a word access that:
 - Accesses either or both bytes of [DWT_COMP m]:[(DWT_COMP m)+1].
 - Transfers a value in which bits[15:0], bits[23:8], or bits[31:16] match DWT_COMP n , even if this value is not the value at memory address DWT_COMP m .

The following pseudocode shows the comparator behavior for inexact data matching:

```
// DataValueMatch
// =====

boolean DataValueMatch(integer N, bits(32) Daddr, integer size, boolean read)

    assert N < UInt(DWT_CTRL.NUMCOMP);
    assert size == 1 || size == 2 || size == 4;

    if DWT_FUNCTION[N].FUNCTION == '0000' || DWT_FUNCTION[N].DATAVMATCH == '0' then
        valid = FALSE;
    elseif N == 0 && DWT_FUNCTION[N].CYCMATCH == '1' then
        UNPREDICTABLE;
    else
        case DWT_FUNCTION[N].FUNCTION of
            when '0101', '1001' // Reads
                valid = read;
            when '0110', '1010' // Writes
                valid = !read;
            when '0111', '1111' // Reads and writes
                valid = TRUE;
            otherwise // Reserved
                UNPREDICTABLE;

        if valid then
            bits(8*size) data = MemU[Daddr, size];
            // The value passed to the DWT must be in little-endian format.

            case DWT_FUNCTION[N].DATAVSZ of
                when '00'
                    case size of
                        when 1
                            data_match = (DWT_COMP[N]<7:0> == data<7:0>);
                        when 2
                            data_match = (DWT_COMP[N]<7:0> == data<7:0> ||
                                DWT_COMP[N]<7:0> == data<15:8>);
                        when 4
                            data_match = (DWT_COMP[N]<7:0> == data<7:0> ||
                                DWT_COMP[N]<7:0> == data<15:8> ||
                                DWT_COMP[N]<7:0> == data<23:16> ||
                                DWT_COMP[N]<7:0> == data<31:24>);
                when '01'
                    case size of
```

```

        when 1
            data_match = FALSE;
        when 2
            data_match = (DWT_COMP[N]<15:0> == data<15:0>);
        when 4
            data_match = (DWT_COMP[N]<15:0> == data<15:0> ||
                           DWT_COMP[N]<15:0> == data<23:8> ||
                           DWT_COMP[N]<15:0> == data<31:16>);
    when '10'
        case size of
            when 1,2
                data_match = FALSE;
            when 4
                data_match = (DWT_COMP[N]<31:0> == data<31:0>);

    when '11'
        data_match = boolean UNKNOWN;

doaddrmatch1 = (DWT_FUNCTION[N].LNK1ENA == '1' &&
                UInt(DWT_FUNCTION[N].DATAVADDR1) != N);
if doaddrmatch1 then
    if UInt(DWT_FUNCTION[N].DATAVADDR1) >= UInt(DWT_CTRL.NUMCOMP) then
        UNPREDICTABLE;
    addrmatch1 = DataAddressMatch(UInt(DWT_FUNCTION[N].DATAVADDR1),
                                   Daddr, size, read, TRUE);
else
    addrmatch1 = FALSE;

doaddrmatch2 = (UInt(DWT_FUNCTION[N].DATAVADDR0) != N);
if doaddrmatch2 then
    if UInt(DWT_FUNCTION[N].DATAVADDR0) >= UInt(DWT_CTRL.NUMCOMP) then
        UNPREDICTABLE;
    addrmatch2 = DataAddressMatch(UInt(DWT_FUNCTION[N].DATAVADDR0),
                                   Daddr, size, read, TRUE);
else
    addrmatch2 = FALSE;

if doaddrmatch1 || doaddrmatch2 then
    match = (addrmatch1 || addrmatch2) && data_match;
else
    match = data_match;
else
    match = FALSE;

return match;

```

LinkAddr support

If a comparator supports and is configured for data value comparisons, it has two options for when it tests for a match:

- On all data accesses, or all read or write accesses if it is configured for RO or WO comparisons.
- Only if the address of the access matches the address specified by another comparator, or in either of two other comparators. This is the LinkAddr option.

If comparator n is a comparator that supports and is configured for a data value comparison, software can set the `DWT_FUNCTION $n$ .DATAVADDR0` field to specify the number of a comparator that defines an address match to use with the data match. Only if both comparators match does the DWT unit perform the action specified by the `DWT_FUNCTION $n$ .FUNCTION` field.

In addition, if the `DWT_FUNCTION $n$ .LNK1ENA` bit is RAO, software can set the `DWT_FUNCTION $n$ .DATAVADDR1` field to specify the number of another comparator that defines a second address match to use with the data match. In this case the DWT unit perform the action specified by the `DWT_FUNCTION $n$ .FUNCTION` field if the data comparator matches and either of the address comparators matches.

Note

For a comparator that supports data address comparison, DWT_FUNCTION n .LNK1ENA is a RO bit. For any other comparator, DWT_FUNCTION m .LNK1ENA is RAZ/WI.

If m is a comparator that defines a linked address comparison for use with a data value comparison, software must program the DWT_FUNCTION m .FUNCTION field to zero. It can also program DWT_MASK m .MASK to a non-zero value, so that the linked address comparator matches on a range of address values.

When comparator n supports and is configured for a data value comparison, and the DWT_FUNCTION n .LNK1ENA bit is RAZ, the programming options are:

To match on any access of the specified type, RO, WO, or RW

Program DWT_FUNCTION n .DATAVADDR0 to n .

To match on an access of the specified type only if an address comparison also matches

Program DWT_FUNCTION n .DATAVADDR0 to m , the number of the comparator that specifies the required address match.

When comparator n supports and is configured for a data value comparison, and the DWT_FUNCTION n .LNK1ENA bit is RAO, the programming options are:

To match on any access of the specified type, RO, WO, or RW

Program both DWT_FUNCTION n .DATAVADDR0 and DWT_FUNCTION n .DATAVADDR1 to n .

To match on an access of the specified type only if a single address comparison also matches

Program both DWT_FUNCTION n .DATAVADDR0 and DWT_FUNCTION n .DATAVADDR1 to m , the number of the comparator that specifies the required address match.

To match on an access of the specified type if either of two address comparisons also matches

Program DWT_FUNCTION n .DATAVADDR0 to m , the number of the comparator that specifies the first possible address match.

Program DWT_FUNCTION n .DATAVADDR1 to p , the number of the comparator that specifies the second possible address match.

See [Comparator behavior for data address matching on page C1-723](#) for more information.

Data trace packet generation

Some DWT comparator functions program the comparator to generate one or more Data trace packets on a successful comparison. [Data trace packets discriminator IDs 8-23 on page D4-793](#) describes the format of these packets. When DWT_CTRL.NOTRCPKT is RAO, the effect of a successful comparison by a comparator configured to generate a Data trace packet is UNPREDICTABLE.

When a comparator function generates a Data trace PC value packet, the PC value returned in the packet is the address of the instruction that made the access.

When a comparator function generates a Data trace address packet, the packet holds bits[15:0] of the matched data address.

Watchpoint debug event generation

When a successful comparison match generates a watchpoint debug event this either:

- Generates a DebugMonitor exception.
- Halts execution, causing the processor to enter Debug state. The address of the next instruction to execute on exiting Debug state, the DebugReturnAddress, is IMPLEMENTATION DEFINED, but must be an instruction that, in a simple sequential execution of the program, the processor would execute after the instruction that triggered the watchpoint. For more information see [Debug Core Register Selector Register, DCRSR on page C1-703](#)

Note

If both Halting debug and the DebugMonitor exception are disabled, the processor ignores the watchpoint debug event.

A PC or data address watchpoint occurs on comparing the PC value or data address with the DWT_COMP n value, and can use a DWT_MASK n value to match on a range of addresses. A data value watchpoint matches only on a specific value of the specified size, and cannot use DWT_MASK n .

A watchpoint debug event is asynchronous to the instruction that caused it. The exception model treats debug watchpoint events in the same way it treats interrupts.

CMPMATCH[N] event generation

Generating a **CMPMATCH[N]** event means the DWT unit uses **CMPMATCH[N]** to signal the event. If the implementation includes an ETM then **CMPMATCH[N]** is an input to the ETM. Other than its connection to an ETM, use of **CMPMATCH[N]** is IMPLEMENTATION DEFINED, and an implementation might provide external **CMPMATCH[N]** pins. If DWT_CTRL.NOEXTTRIG is RAO, the effect of a successful comparison by a comparator configured to generate a **CMPMATCH[N]** event is UNPREDICTABLE.

Checking the implemented features of the DWT comparators

Most features of the DWT comparators are optional. This section summarizes how software can check what features the implemented comparators support.

Implementation of comparators, and number of comparators

The DWT_CTRL.NUMCOMP field indicates the number of implemented comparators. The value is zero if the DWT unit does not support comparators.

Note

All implemented comparators must support address comparisons.

Support for cycle count comparison, comparator 0 only

The DWT_CTRL.NOCYCCNT bit indicates whether comparator 0 supports cycle count comparisons. If this bit is RAZ, comparator 0 supports cycle count comparisons.

Note

Cycle count comparison is possible only when the cycle counter is enabled. Software sets DWT_CTRL.CYCCNTENA to 1 to enable the cycle counter.

Support for data value comparison

To check whether comparator n supports data value comparison, software must write 1 to the DWT_FUNCTION n .DATAVMATCH bit, and then read the DWT_FUNCTION n register and check whether the bit reads as 1. If it does, the comparator supports data value comparison.

The comparators that support data value comparison, if any, start at comparator 1 and number upwards sequentially. Comparator 0 might support data value comparison if all other comparators support data value comparison.

Number of linked address comparators, for a comparator that supports data value comparison

If comparator n supports data value comparison, the DWT_FUNCTION n .LNK1ENA bit indicates whether the comparator supports one or two linked address comparators:

- If LNK1ENA is 0, the comparator supports only one linked address comparator, specified by the DATAVADDR0 field. The DATAVADDR1 field is RAZ/WI.
- If LNK1ENA is 1, the comparator supports two linked address comparators, specified by the DATAVADDR0 and DATAVADDR1 fields.

Maximum size of the address comparison mask

To find the maximum size of the address comparison mask for comparator n , software must write 0b11111 to the DWT_MASK n .MASK field, and then read the DWT_MASK n register. The value returned in the MASK field is the maximum mask size.

Generation of Data trace packets

The DWT_CTRL.NOTRCPKT bit indicates whether the DWT supports generation of Data trace packets. If this bit is 0, the DWT can generate these packets. The comparators can generate Data trace packets only if the cycle counter is implemented and enabled, indicated by DWT_CTRL.NOCYCCNT being RAZ and DWT_CTRL.CYCCNTENA being set to 1.

In addition, the ITM transmits DWT trace packets only if software sets the ITM_TCR.TXENA bit to 1.

Generation of CMPMATCH[N] events

The DWT_CTRL.NOEXTTRIG bit indicates whether the DWT supports generation of CMPMATCH[N] events. If this bit is RAZ, the DWT can generate these events.

For descriptions of the registers referred to in this subsection see:

- [Control register, DWT_CTRL on page C1-737](#).
- [Comparator Function registers, DWT_FUNCTION \$n\$ on page C1-746](#).
- [Comparator Mask registers, DWT_MASK \$n\$ on page C1-745](#).
- [Trace Control Register, ITM_TCR on page C1-716](#).

C1.8.2 Exception trace support

Exception trace support is an optional debug feature. The DWT_CTRL.NOTRCPKT bit is RAZ if the implementation includes exception trace support, see [Control register, DWT_CTRL on page C1-737](#).

If it is supported, software enables exception tracing by setting the DWT_CTRL.EXCTRCENA bit to 1, see [Control register, DWT_CTRL on page C1-737](#). When exception tracing is enabled, the DWT generates an Exception trace packet when any of the following occurs:

- The processor enters an exception handler, from Thread mode or by preemption of a thread or handler. See [Exception entry behavior on page B1-531](#).
- The processor exits an exception handler with an EXC_RETURN vector. See [Exception return behavior on page B1-539](#).
- The processor returns from an exception, reentering a preempted thread or handler code sequence.

For more information about the exception trace packet see [Exception trace packets, discriminator ID1 on page D4-791](#).

C1.8.3 CYCCNT cycle counter and related timers

CYCCNT is an optional free-running 32-bit cycle counter. If the DWT unit implements CYCCNT then the DWT_CTRL.NOCYCCNT bit is RAZ, see [Control register, DWT_CTRL on page C1-737](#).

When implemented and enabled, CYCCNT increments on each cycle of the processor clock. When the counter overflows it wraps to zero, transparently.

CYCCNT does not increment when the processor is halted in Debug state.

The DWT_CTRL.CYCCNTENA bit enables the CYCCNT counter. Software can access the DWT_CYCCNT register to read the current value of CYCCNT, or to set the CYCCNT value, see [Cycle Count register, DWT_CYCCNT on page C1-741](#).

The DWT unit obtains two other timers from CYCCNT, see:

- [The POSTCNT timer on page C1-732](#).
- [The synchronization packet timer on page C1-733](#).

Note

Software that uses the CYCCNT counter for profiling must be aware that:

- When CYCCNT overflows it wraps transparently to zero.
 - Disabling or enabling CYCCNT, or changing its value, affects the POSTCNT and synchronization packet timers.
-

The POSTCNT timer

POSTCNT is a 4-bit countdown counter derived from CYCCNT, that acts as a timer for periodic generation of Periodic PC sample packets or Event counter packets, when these packets are enabled.

Note

Periodic PC sample packets are not the same as the Data trace PC value packets that are generated by the DWT comparators. For the differences in the packet formats, see:

- [Periodic PC sample packets, discriminator ID2 on page D4-792.](#)
 - [Data trace PC value packet format on page D4-794.](#)
-

The DWT does not support generation of Periodic PC sample packets or Event counter packets if:

- It does not implement the CYCCNT timer.
- DWT_CTRL.NOTRCPKT is RAO, see [Control register, DWT_CTRL on page C1-737.](#)

The DWT_CTRL.CYCTAP bit selects the CYCCNT tap bit for POSTCNT, see [Control register, DWT_CTRL on page C1-737.](#) [Table C1-17](#) shows the effect of this bit:

Table C1-17 CYCCNT tap bit for POSTCNT timer

CYCTAP bit	CYCCNT tap at	POSTCNT clock rate
0	Bit[6]	(Processor clock)/64
1	Bit[10]	(Processor clock)/1024

When software enables use of the POSTCNT timer, the processor loads the initial POSTCNT value from the DWT_CTRL.POSTINIT field.

Subsequently whenever the CYCCNT tap bit transitions, either from 0 to 1 or from 1 to 0:

- If POSTCNT holds a nonzero value, POSTCNT decrements by 1.
- If POSTCNT is zero, the DWT unit:
 - Reloads POSTCNT from DWT_CTRL.POSTPRESET.
 - Generates the required Periodic PC sample packets or Event counter packet.

The DWT_CTRL.PCSAMPLENA enables Periodic PC sample packets, or DWT_CTRL.CYCEVTENA enables POSTCNT underflow Event counter packets. [Table C1-18](#) summarizes how these bits control the operation of the POSTCNT counter.

Table C1-18 Effect of a CYCCNT tap bit transition when POSTCNT is zero

DWT_CTRL bit		Action
PCSAMPLENA	CYCEVTENA	
0	0	POSTCNT disabled.
0	1	On writing these bit values to DWT_CTRL when the bits were previously both 0, load POSTCNT from DWT_POSTINIT, and start POSTCNT counting. Subsequently, whenever POSTCNT underflows from zero: - Reload POSTCNT from POSTPRESET. - Generate Event counter packet with the Cyc bit set to 1, see Event counter packet, discriminator ID0 on page D4-790.
1	0	On writing these bit values to DWT_CTRL when the bits were previously both 0, load POSTCNT from DWT_POSTINIT, and start POSTCNT counting. Subsequently, whenever POSTCNT underflows from zero: - Reload POSTCNT from POSTPRESET. - Generate Periodic PC sample packet, see Periodic PC sample packets, discriminator ID2 on page D4-792
1	1	Arm deprecates the use of this combination of DWT_CTRL bit values. In early Armv7-M implementations, setting both PCSAMPLENA and CYCEVTENA to 1 has the same effect as setting PCSAMPLENA to 1 and CYCEVTENA to 0. Arm does not guarantee future Armv7-M implementations will behave in this way.

Note

- The enable bit for the POSTCNT counter underflow event is DWT_CTRL.CYCEVTENA. There is no overflow event for the CYCCNT counter. When CYCCNT overflows it wraps to zero transparently.
- Software cannot access the POSTCNT value directly, or change this value.

Software must write to DWT_CTRL.POSTINIT to define the required initial value of the POSTCNT counter, and then perform a second write to DWT_CTRL to set either the PCSAMPLENA bit or the CYCEVTENA bit to 1, to enable the POSTCNT counter. For more information see [Control register, DWT_CTRL on page C1-737](#).

Disabling CYCCNT stops POSTCNT. Enabling CYCCNT also loads POSTCNT from the DWT_CTRL.POSTINIT field.

The synchronization packet timer

A tap on the CYCCNT counter provides a timer signal for generation of periodic Synchronization packets by the ITM. The DWT_CTRL.SYNCTAP field determines the position of this tap, and therefore the Synchronization packet rate. [Table C1-19](#) shows the effect of this field.

Table C1-19 CYCCNT tap bit for synchronization packet timer

SYNCTAP field	CYCCNT tap at	Synchronization packet rate
0b00	-	Synchronization packet timer disabled

Table C1-19 CYCCNT tap bit for synchronization packet timer (continued)

SYNCTAP field	CYCCNT tap at	Synchronization packet rate
0b01	Bit[24]	(Processor clock)/16M
0b10	Bit[26]	(Processor clock)/64M
0b11	Bit[28]	(Processor clock)/256M

The ITM generates periodic Synchronization packets only if both:

- DWT_CTRL.SYNCTAP is non-zero, see [Control register, DWT_CTRL on page C1-737](#).
- ITM_TCR.SYNCENA is set to 1, see [Trace Control Register, ITM_TCR on page C1-716](#).

When DWT_CTRL.SYNCTAP is non-zero, the synchronization packet timer generates a pulse every time the CYCCNT tap bit transitions, either from 1 to 0 or from 0 to 1. When this happens:

- If ITM_TCR.SYNCENA is set to 1, the ITM generates a Synchronization packet.
- If the processor implements global timestamps, the DWT signals a request for a full timestamp, see [Global timestamping on page C1-711](#).

For more information about Synchronization packets see [Synchronization support on page C1-712](#) and [Synchronization packet on page D4-782](#).

C1.8.4 Profiling counter support

Profiling counter support is an optional debug feature. The DWT_CTRL.NOPRFCNT bit is RAZ if these counters are implemented.

If the implementation includes profiling counter support, the DWT unit provides the following 8-bit event counters, for software profiling:

- A general counter for instruction cycle count estimation. See [CPI Count register, DWT_CPICNT on page C1-741](#).
- An exception overhead counter. See [Exception Overhead Count register, DWT_EXCCNT on page C1-742](#).
- A sleep overhead counter. See [Sleep Count register, DWT_SLEPCNT on page C1-742](#).
- A load-store counter. See [LSU Count register, DWT_LSUCNT on page C1-743](#).
- A folded instruction counter. See [Folded-instruction Count register, DWT_FOLDCNT on page C1-744](#).

If profiling counter support is implemented the following formula must hold:

$$\text{ICNT} = \text{CNT}_{\text{CYCLES}} + \text{CNT}_{\text{FOLD}} - (\text{CNT}_{\text{LSU}} + \text{CNT}_{\text{EXC}} + \text{CNT}_{\text{SLEEP}} + \text{CNT}_{\text{CPI}})$$

Where:

ICNT is the total number of instructions architecturally executed.

CNT_{CYCLES} is the number of cycles counted by DWT_CYCCNT.

CNT_{FOLD} is the number of instructions counted by DWT_FOLDCNT.

CNT_{LSU} is the number of cycles counted by DWT_LSUCNT.

CNT_{EXC} is the number of cycles counted by DWT_EXCCNT.

CNT_{SLEEP} is the number of cycles counted by DWT_SLEPCNT.

CNT_{CPI} is the number of cycles counted by DWT_CPICNT.

A counter overflows on every 256th cycle counted. When this happens, the counter wraps to 0, and if the appropriate counter overflow event is enabled in the DWT_CTRL register, the DWT outputs an Event counter packet with the appropriate counter flag set to 1. [Table C1-20](#) shows the DWT_CTRL counter overflow event enable bit for each of the counters. Setting one of these bits to 1 also clears the corresponding counter to zero. For more information, see [Control register, DWT_CTRL on page C1-737](#).

Table C1-20 DWT_CTRL profile counter overflow event enable bits

Counter	DWT_CTRL counter overflow event enable bit
DWT_CPICNT	CPIEVTENA, bit[17]
DWT_EXCCNT	EXCEVTENA, bit[18]
DWT_SLEEPCNT	SLEEPEVTENA, bit[19]
DWT_LSUCNT	LSUEVTENA, bit[20]
DWT_FOLDCNT	FOLDEVTENA, bit[21]

For more information about the Event counter packet see [Event counter packet, discriminator ID0 on page D4-790](#).

Profiling counter accuracy

The counters provide approximately accurate performance count information, but to keep the implementation and validation cost low, the architecture accepts a reasonable degree of inaccuracy in the counts is acceptable. The architecture does not define a reasonable degree of inaccuracy, but Arm gives the following guidelines:

- Under normal operating conditions, the counters present an accurate value of the overall system cycle counts.
- Although the counters can be used with Halting debug, they are intended for non-intrusive operation. Entry to or exit from Debug state can be a source of inaccuracy. Counters do not increment when the processor is halted. In Debug state, the overhead associated with STEP and RUN commands from and to the halt condition is IMPLEMENTATION DEFINED.
- Arm strongly recommends that, when an instruction is used to enter or exit an exception or sleep state, the cycle count associated with the instruction is minimal, and the remaining cycles are associated with the exception overhead or with the energy-saving state. However, the exact division is IMPLEMENTATION DEFINED. Examples of the instructions this refers to are SVC and WFI.
- In superscalar implementations FOLD counts can be very high, affecting profiling statistics. Profile data validity normally improves when aggregated over a longer time, with a large working set of data and instructions.

The fact that the architecture permits inaccuracy limits the possible uses of the performance counters. In particular, the architecture does not define the point in a pipeline where a particular instruction increments a performance counter, relative to the point where software can read the performance counter. Therefore, pipelining can add some imprecision. Profile counter size and the DWT event generation model are designed for non-intrusive operation, where the DWT generates information for remote tracing, processing and analysis, without the system overhead of software reads and processing by the processor itself.

An implementation must document any scenarios where significant inaccuracies are expected.

C1.8.5 Program counter sampling support

The DWT_PCSR is an optional component of an Armv7-M debug implementation, see [Program Counter Sample Register, DWT_PCSR on page C1-745](#). If an implementation does not include the DWT_PCSR, the DWT_PCSR location is RAZ/WI.

———— Note ————

DWT_PCSR program counter sampling is independent of the PC sampling provided by:

- Periodic PC sample packets, described in [The POSTCNT timer on page C1-732](#).

- Data trace PC value packets generated as a result of a DWT comparator match, see [The DWT comparators on page C1-719](#).

A debugger can read DWT_PCSR without changing the behavior of any code executing on the processor. This provides a mechanism for coarse-grained non-intrusive profiling of software execution. When DWT_PCSR is implemented, a read of the register returns one of the following:

- The address of an instruction recently executed by the processor.
- 0xFFFFFFFF if the processor is in Debug state, or in a state and mode that do not permit non-invasive debug.

Note

- The architecture does not define recently executed, and does not define the delay between an instruction being executed by the processor and its address appearing in the DWT_PCSR. For example, if code reads the DWT_PCSR of the processor it is running on, the architecture does not guarantee any relationship between the value returned by the DWT_PCSR read and the program counter value corresponding to that piece of code. The DWT_PCSR is intended for use only by an external agent to provide statistical information for code profiling. A read of the DWT_PCSR by the processor can return an UNKNOWN value.
- A debug agent must not rely on a return value of 0xFFFFFFFF to indicate that the processor is halted. It must interrogate the S_HALT bit in the DHCSR for this purpose, see [Debug Halting Control and Status Register, DHCSR on page C1-700](#).

When DWT_PCSR returns a value other than 0xFFFFFFFF, the returned value always references an instruction that has been committed for execution. It is IMPLEMENTATION DEFINED whether an instruction that failed its condition codes is considered as a committed instruction, but Arm recommends that these instructions are considered as committed instructions. A read of DWT_PCSR must not return the address of an instruction that has been fetched but not committed for execution.

When DWT_PCSR is implemented, it must be able to sample references to branch targets. It is IMPLEMENTATION DEFINED whether it can sample references to other instructions. Arm recommends that it can sample a reference to any instruction.

The branch target for a conditional branch that fails its condition code is the instruction that immediately follows the conditional branch instruction.

When DEMCR.TRCENA is set to 0, any read of DWT_PCSR returns an UNKNOWN value. For more information see [Debug Exception and Monitor Control Register, DEMCR on page C1-706](#).

Note

The DWT_CTRL.PCSAMPLENA bit does not affect PC sampling by DWT_PCSR.

C1.8.6 DWT register summary

[Table C1-21](#) shows the DWT registers in address order. An entry of IMP DEF in the *Reset* column indicates that the reset value of the register is IMPLEMENTATION DEFINED. See the register description for more information. All registers are 32-bits wide.

Table C1-21 DWT register summary

Address	Name	Type	Reset	Description
0xE0001000	DWT_CTRL	RW	IMP DEF	Control register, DWT_CTRL on page C1-737 .
0xE0001004	DWT_CYCCNT	RW	UNKNOWN	Cycle Count register, DWT_CYCCNT on page C1-741 .
0xE0001008	DWT_CPICNT	RW	UNKNOWN	CPI Count register, DWT_CPICNT on page C1-741 .
0xE000100C	DWT_EXCCNT	RW	UNKNOWN	Exception Overhead Count register, DWT_EXCCNT on page C1-742 .

Table C1-21 DWT register summary (continued)

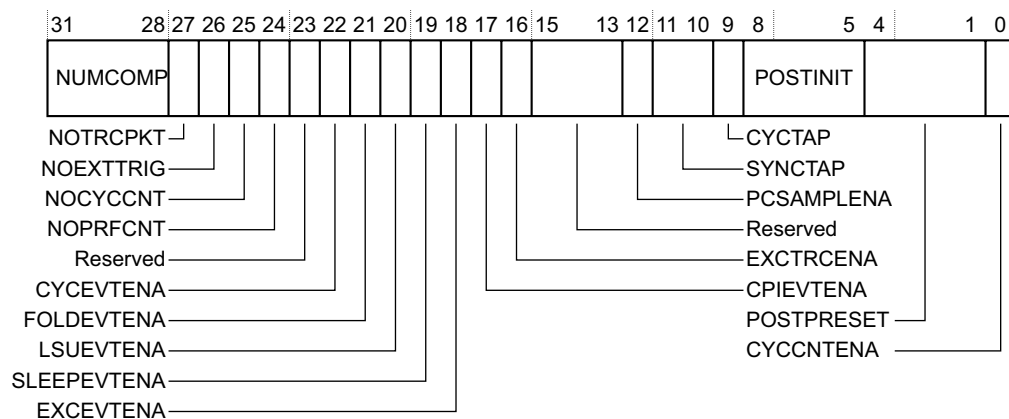
Address	Name	Type	Reset	Description
0xE0001010	DWT_SLEPCNT	RW	UNKNOWN	<i>Sleep Count register, DWT_SLEPCNT on page C1-742.</i>
0xE0001014	DWT_LSUCNT	RW	UNKNOWN	<i>LSU Count register, DWT_LSUCNT on page C1-743.</i>
0xE0001018	DWT_FOLDCNT	RW	UNKNOWN	<i>Folded-instruction Count register, DWT_FOLDCNT on page C1-744.</i>
0xE000101C	DWT_PCSR	RO	UNKNOWN	<i>Program Counter Sample Register, DWT_PCSR on page C1-745.</i>
The block of registers addressed at 0xE0001020-0xE000102C repeat for each comparator, starting at comparator 0. <i>n</i> takes each value from 0 to (DWT_CTRL.NUMCOMP-1).				
0xE0001020 + 16 <i>n</i>	DWT_COMP <i>n</i>	RW	UNKNOWN	<i>Comparator registers, DWT_COMP<i>n</i> on page C1-745.</i>
0xE0001024 + 16 <i>n</i>	DWT_MASK <i>n</i>	RW	UNKNOWN	<i>Comparator Mask registers, DWT_MASK<i>n</i> on page C1-745.</i>
0xE0001028 + 16 <i>n</i>	DWT_FUNCTION <i>n</i>	RW	See Description	<i>Comparator Function registers, DWT_FUNCTION<i>n</i> on page C1-746.</i>
0xE000102C + 16 <i>n</i>	-	-	-	Reserved
0xE0001F00-0xE0001FFC	-	RO	IMP DEF	Optional CoreSight management and ID registers. See Appendix D1 Armv7-M CoreSight Infrastructure IDs for more information.

C1.8.7 Control register, DWT_CTRL

The DWT_CTRL register characteristics are:

Purpose	Provides configuration and status information for the DWT unit, and used to control features of the unit.
Usage constraints	There are no usage constraints.
Configurations	Always implemented.
Attributes	See Table C1-21 on page C1-736 and the register field descriptions.

The DWT_CTRL register bit assignments are:



NUMCOMP, bits[31:28]

Number of comparators implemented.

A value of zero indicates no comparator support.

These bits are read-only. The reset value is IMPLEMENTATION DEFINED.

NOTRCPKT, bit[27]

Shows whether the implementation supports trace sampling and exception tracing:

- 0** Trace sampling and exception tracing supported.
- 1** Trace sampling and exception tracing not supported.

If this bit is RAZ, the NOCYCCNT bit must also RAZ.

This bit is read-only. The reset value is IMPLEMENTATION DEFINED.

NOEXTTRIG, bit[26]

Shows whether the implementation includes external match signals, **CMPMATCH[N]**:

- 0** **CMPMATCH[N]** supported.
- 1** **CMPMATCH[N]** not supported.

This bit is read-only. The reset value is IMPLEMENTATION DEFINED.

NOCYCCNT, bit[25]

Shows whether the implementation supports a cycle counter:

- 0** Cycle counter supported.
- 1** Cycle counter not supported.

For more information see [CYCCNT cycle counter and related timers on page C1-731](#).

This bit is read-only. The reset value is IMPLEMENTATION DEFINED.

NOPRFCNT, bit[24]

Shows whether the implementation supports the profiling counters:

- 0** Supported.
- 1** Not supported.

For more information see [Profiling counter support on page C1-734](#).

This bit is read-only. The reset value is IMPLEMENTATION DEFINED.

Bits[23]

Reserved.

CYCEVTENA, bit[22]

Enables POSTCNT underflow Event counter packets generation:

- 0** No POSTCNT underflow packets generated.
- 1** POSTCNT underflow packets generated, if PCSAMPLENA set to 0.

See [The POSTCNT timer on page C1-732](#) for more information.

This bit is UNK/SBZP if the NOTRCPKT bit is RAO or the NOCYCCNT bit is RAO. The reset value is 0.

FOLDEVTENA, bit[21]

Enables generation of the Folded-instruction counter overflow event:

- 0** Disabled.
- 1** Enabled.

This bit is UNK/SBZP if the NOPRFCNT bit is RAO. The reset value is 0.

LSUEVTENA, bit[20]

Enables generation of the LSU counter overflow event.

- 0** Disabled.
- 1** Enabled.

This bit is UNK/SBZP if the NOPRFCNT bit is RAO. The reset value is 0.

SLEEPEVTENA, bit[19]

Enables generation of the Sleep counter overflow event.

- 0** Disabled.

1 Enabled.

This bit is UNK/SBZP if the NOPRFCNT bit is RAO. The reset value is 0.

EXCEVTENA, bit[18]

Enables generation of the Exception overhead counter overflow event:

0 Disabled.

1 Enabled.

This bit is UNK/SBZP if the NOPRFCNT bit is RAO. The reset value is 0.

CPIEVTENA, bit[17]

Enables generation of the CPI counter overflow event:

0 Disabled.

1 Enabled.

This bit is UNK/SBZP if the NOPRFCNT bit is RAO. The reset value is 0.

EXCTRCENA, bit[16]

Enables generation of exception trace:

0 Disabled.

1 Enabled.

This bit is UNK/SBZP if the NOTRCPKT bit is RAO. The reset value is 0.

Bits[15:13]

Reserved.

PCSAMPLENA, bit[12]

Enables use of POSTCNT counter as a timer for Periodic PC sample packet generation:

0 No Periodic PC sample packets generated.

1 Periodic PC sample packets generated.

See [The POSTCNT timer on page C1-732](#) for more information.

This bit is UNK/SBZP if the NOTRCPKT bit is RAO or the NOCYCCNT bit is RAO. The reset value is 0.

SYNCTAP, bits[11:10]

Selects the position of the synchronization packet counter tap on the CYCCNT counter. This determines the Synchronization packet rate:

00 Disabled. No Synchronization packets.

01 Synchronization counter tap at CYCCNT[24].

10 Synchronization counter tap at CYCCNT[26].

11 Synchronization counter tap at CYCCNT[28].

For more information see [The synchronization packet timer on page C1-733](#).

This field is UNK/SBZP if the NOCYCCNT bit is RAO. The reset value is UNKNOWN.

CYCTAP, bit[9]

Selects the position of the POSTCNT tap on the CYCCNT counter:

0 POSTCNT tap at CYCCNT[6].

1 POSTCNT tap at CYCCNT[10].

For more information see [The POSTCNT timer on page C1-732](#).

This bit is UNK/SBZP if the NOCYCCNT bit is RAO. The reset value is UNKNOWN.

POSTINIT, bits[8:5]

Initial value for the POSTCNT counter. For more information see [Enabling POSTCNT, and behavior of accesses to the DWT_CTRL.POSTINIT field on page C1-740](#) and [The POSTCNT timer on page C1-732](#).

This field is UNK/SBZP if the NOCYCCNT bit is RAO. The reset value is UNKNOWN.

Note

This field was previously called POSTCNT. The changed name gives a better indication of its function.

POSTPRESET, bits[4:1]

Reload value for the POSTCNT counter. For more information see [The POSTCNT timer on page C1-732](#).

This field is UNK/SBZP if the NOCYCCNT bit is RAO. The reset value is UNKNOWN.

CYCCNTENA, bit[0] Enables CYCCNT:

0	Disabled.
1	Enabled.

This bit is UNK/SBZP if the NOCYCCNT bit is RAO. The reset value is 0.

Enabling POSTCNT, and behavior of accesses to the DWT_CTRL.POSTINIT field

Before enabling the POSTCNT counter, software must write the required initial value of the counter to the DWT_CTRL.POSTINIT field. Then it must perform a second write to DWT_CTRL, to set either DWT_CTRL.CYCEVTENA or DWT_CTRL.PCSAMPLENA to 1, to enable the POSTCNT counter.

The processor ignores any write to DWT_CTRL.POSTINIT field if the POSTCNT counter is enabled. That is, it ignores a write to this field unless, before the write, DWT_CTRL.CYCEVTENA and DWT_CTRL.PCSAMPLENA are both 0.

For any write to the DWT_CTRL register that changes the values of DWT_CTRL.CYCEVTENA and DWT_CTRL.PCSAMPLENA bits, other than a change from both bits being zero to exactly one of the bits being one, it is UNPREDICTABLE whether POSTCNT is reloaded from DWT_CTRL.POSTINIT or left unchanged.

Note

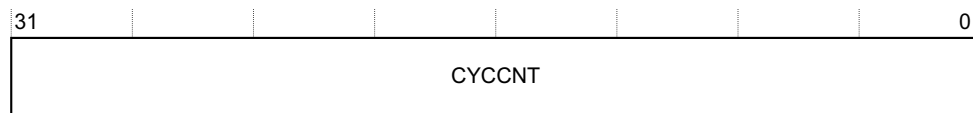
This UNPREDICTABLE behavior does not matter when setting both DWT_CTRL.CYCEVTENA and DWT_CTRL.PCSAMPLENA to zero, to disable POSTCNT, because it does not affect the behavior on re-enabling POSTCNT.

C1.8.8 Cycle Count register, DWT_CYCCNT

The DWT_CYCCNT register characteristics are:

Purpose	Shows or sets the value of the processor cycle counter, CYCCNT.
Usage constraints	The DWT unit suspends CYCCNT counting when the processor is in Debug state.
Configurations	Implemented only when DWT_CTRL.NOCYCCNT is RAZ, see Control register, DWT_CTRL on page C1-737 . When DWT_CTRL.NOCYCCNT is RAO no cycle counter is implemented and this register is UNK/SBZP.
Attributes	See Table C1-21 on page C1-736 .

The DWT_CYCCNT register bit assignments are:



CYCCNT, bits[31:0] Incrementing cycle counter value. When enabled, CYCCNT increments on each processor clock cycle when DWT_CTRL.CYCNTENA == 1 and DEMCR.TRCENA == 1. On overflow, CYCCNT wraps to zero.

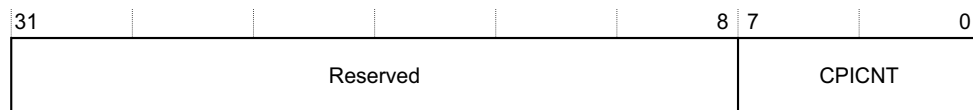
For more information see [CYCCNT cycle counter and related timers on page C1-731](#).

C1.8.9 CPI Count register, DWT_CPICNT

The DWT_CPICNT register characteristics are:

Purpose	Counts additional cycles required to execute multicycle instructions and instruction fetch stalls.
Usage constraints	The counter initializes to 0 when software enables its counter overflow event by setting the DWT_CTRL.CPIEVTENA bit to 1.
Configurations	Implemented only when DWT_CTRL.NOPRFCNT is RAZ, see Control register, DWT_CTRL on page C1-737 . If DWT_CTRL.NOPRFCNT is RAO, indicating that the implementation does not include the profiling counters, this register is UNK/SBZP.
Attributes	See Table C1-21 on page C1-736 .

The DWT_CPICNT register bit assignments are:



Bits[31:8] Reserved, UNK/SBZP.

CPICNT, bits[7:0] Base instruction overhead counter. Counts one on each cycle when all of the following are true:

- DWT_CTRL.CPIEVTENA == 1 and DEMCR.TRCENA == 1.
- No instruction is executed.
- No load-store operation is in progress, see [LSU Count register, DWT_LSUCNT on page C1-743](#).
- No exception-entry or exception-exit operation is in progress, see [Exception Overhead Count register, DWT_EXCCNT on page C1-742](#).
- Not in a power-saving mode, see [Sleep Count register, DWT_SLEEPCNT](#).

- Non-invasive debug is enabled by the authentication interface, see [Debug authentication on page C1-688](#).

The definition of “no instruction is executed” is IMPLEMENTATION DEFINED. Arm recommends that this counts each cycle on which no instruction is retired.

DWT_CPICNT is initialized to zero when the counter is disabled and DWT_CTRL.CPIEVTENA transitions from 0 to 1. An Event Counter packet is emitted on counter overflow.

For more information see [Profiling counter support](#) on page C1-734.

C1.8.10 Exception Overhead Count register, DWT EXCCNT

The DWT_EXCCNT register characteristics are:

Purpose	Counts the total cycles spent in exception processing.
Usage constraints	The counter initializes to 0 when software enables its counter overflow event by setting the DWT_CTRL.EXCEVTENA bit to 1.
Configurations	Implemented only when DWT_CTRL.NOPRFCNT is RAZ, see Control register, DWT_CTRL on page C1-737 . If DWT_CTRL.NOPRFCNT is RAO, indicating that the implementation does not include the profiling counters, this register is UNK/SBZP.
Attributes	See Table C1-21 on page C1-736 .

The DWT_EXCCNT register bit assignments are:

31							8	7			0
Reserved								EXCCNT			

Bits[31:8]	Reserved, UNK/SBZP.
EXCCNT, bits[7:0]	The exception overhead counter. Counts one on each cycle when all of the following are true: • $DWT_CTRL.EXCEVTENA = 1$ and $DEMCR.TRCENA = 1$. • No instruction is executed, see CPI Count register, DWT_CPICNT on page C1-741. • An exception-entry or exception-exit related operation is in progress. • Non-invasive debug is enabled by the authentication interface, see Debug authentication on page C1-688. Exception-entry or exception-exit related operations include the stacking of registers on exception entry, unstacking of registers on exception exit, and preemption. DWT_EXCCNT is initialized to zero when the counter is disabled and $DWT_CTRL.EXCEVTENA$ transitions from 0 to 1. An Event Counter packet is emitted on counter overflow.

For more information see *Profiling counter support* on page C1-734.

C1.8.11 Sleep Count register, DWT_SLEEPCNT

The DWT SLEPCNT register characteristics are:

Purpose	Counts the total number of cycles that the processor is sleeping.
Usage constraints	The counter initializes to 0 when software enables its counter overflow event by setting the DWT_CTRL.SLEEPEVTENA bit to 1.

Configurations	Implemented only when DWT_CTRL.NOPRFCNT is RAZ, see Control register; DWT_CTRL on page C1-737.
-----------------------	--

If DWT_CTRL.NOPRFCNT is RAO, indicating that the implementation does not include the profiling counters, this register is UNK/SBZP.

Attributes See [Table C1-21](#) on page C1-736.

The DWT_SLEPCNT register bit assignments are:

31								8	7			0
Reserved										SLEEP_CNT		

Bits[31:8]	Reserved, UNK/SBZP.
-------------------	---------------------

SLEPCNT, bits[7:0] Sleep counter. Counts one on each cycle when all of the following are true:

- DWT_CTRL.SLEEPEVTENA == 1 and DEMCR.TRCENA == 1.
- No instruction is executed, see [CPI Count register; DWT_CPICNT](#) on page C1-741.
- No load-store operation is in progress, see [LSU Count register; DWT_LSUCNT](#).
- No exception-entry or exception-exit operation is in progress, see [Exception Overhead Count register; DWT_EXCCNT](#) on page C1-742.
- In a power-saving mode.
- Non-invasive debug is enabled by the authentication interface, see [Debug authentication](#) on page C1-688.

Power-saving modes include WFI, WFE, and Sleep on exit, see [Power management on page B1-559](#).

DWT_SLEEP_CNT is initialized to zero when the counter is disabled and DWT_CTRL.SLEEPEVTENA transitions from 0 to 1. An Event Counter packet is emitted on overflow.

Arm recommends that this counter counts all cycles when the processor is sleeping, regardless of whether a WFI or WFE instruction, or the sleep-on-exit functionality, caused the entry to sleep mode. However, all sleep features are IMPLEMENTATION DEFINED and therefore when this counter counts is IMPLEMENTATION DEFINED.

For more information see *Profiling counter support* on page C1-734.

C1.8.12 LSU Count register, DWT_LSUCNT

The DWT LSUCNT register characteristics are:

Purpose	Increments on the additional cycles required to execute all load or store instructions.
----------------	---

Usage constraints	The counter initializes to 0 when software enables its counter overflow event by setting the DWT_CTRL.LSUEVTENA bit to 1.
--------------------------	---

Configurations	Implemented only when DWT_CTRL.NOPRFCNT is RAZ, see Control register, DWT_CTRL on page C1-737.
-----------------------	--

If DWT_CTRL.NOPRFCNT is RAO, indicating that the implementation does not include the profiling counters, this register is UNK/SBZP.

Attributes See [Table C1-21](#) on page C1-736.

The DWT_LSUCNT register bit assignments are:

31							8	7			0
Reserved								LSUCNT			

Bits[31:8]	Reserved, UNK/SBZP.
-------------------	---------------------

LSUCNT, bits[7:0] Load-store overhead counter. Counts one on each cycle when all of the following are true:

- DWT_CTRL.LSUEVTENA == 1 and DEMCR.TRCENA == 1.
- No instruction is executed, see *CPI Count register, DWT_CPICNT* on page C1-741.
- No exception-entry or exception-exit operation is in progress, see *Exception Overhead Count register, DWT_EXCCNT* on page C1-742.
- A load-store operation is in progress.
- Non-invasive debug is enabled by the authentication interface, see *Debug authentication* on page C1-688.

DWT_LSUCNT is initialized to zero when the counter is disabled and DWT_CTRL.LSUEVTENA transitions from 0 to 1. An Event Counter packet is emitted on counter overflow.

For more information see *Profiling counter support* on page C1-734.

C1.8.13 Folded-instruction Count register, DWT FOLDCNT

The DWT_FOLDCNT register characteristics are:

Purpose	Increments on each instruction that takes 0 cycles.
Usage constraints	- The counter initializes to 0 when software enables its counter overflow event by setting the DWT_CTRL.FOLDEVTENA bit to 1. - If an implementation includes profiling counters but does not support instruction folding, this counter can be RAZ/WI.
Configurations	Implemented only when DWT_CTRL.NOPRFCNT is RAZ, see Control register, DWT_CTRL on page C1-737. If DWT_CTRL.NOPRFCNT is RAO, indicating that the implementation does not include the profiling counters, this register is UNK/SBZP.
Attributes	See Table C1-21 on page C1-736 .

The DWT_FOLDCNT register bit assignments are:

31							8	7			0
Reserved								FOLDCNT			

Bits[31:8]	Reserved, UNK/SBZP.
-------------------	---------------------

FOLDCNT, bits[7:0] Folded instruction counter. Counts on each cycle when at least two instructions are executed. The counter is incremented by the number of instructions executed, minus one.

Counts on each cycle when all of the following are true:

- DWT_CTRL.FOLDEVTENA == 1 and DEMCR.TRCENA == 1.
- At least two instructions are executed, see *CPI Count register, DWT_CPICNT* on page C1-741.
- Non-invasive debug is enabled by the authentication interface, see *Debug authentication* on page C1-688.

DWT_FOLDCNT is initialized to zero when the counter is disabled and DWT_CTRL.FOLDEVTENA transitions from 0 to 1. An Event Counter packet is emitted on counter overflow.

For more information see [Profiling counter support](#) on page C1-734.

C1.8.14 Program Counter Sample Register, DWT_PCSR

The DWT_PCSR characteristics are:

Purpose Samples the current value of the program counter.

Usage constraints There are no usage constraints.

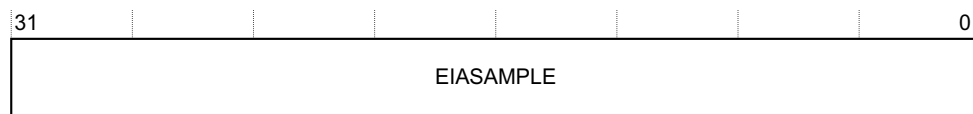
———— Note ————

Bit[0] of any sampled value is RAZ and does not reflect instruction set state as it does in a PC sample on the Armv7-A and Armv7-R architecture profiles.

Configurations An optional feature. Register is RAZ/WI if not implemented.

Attributes See [Table C1-21 on page C1-736](#).

The DWT_PCSR bit assignments are:



EIASAMPLE, bits[31:0]

Executed Instruction Address sample value.

For more information see [Program counter sampling support on page C1-735](#).

C1.8.15 Comparator registers, DWT_COMP n

The DWT_COMP n register characteristics are:

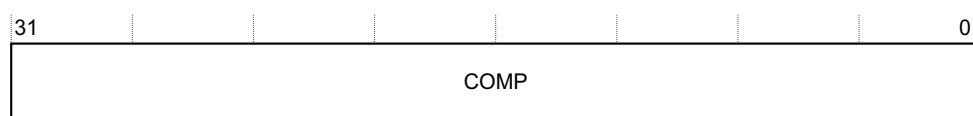
Purpose Provides a reference value for use by comparator n .

Usage constraints The operation of comparator n depends also on the registers DWT_MASK n and DWT_FUNCTION n , see [Comparator Mask registers, DWT_MASK \$n\$](#) and [Comparator Function registers, DWT_FUNCTION \$n\$](#) on page C1-746.

Configurations Implemented only when DWT_CTRL.NUMCOMP is nonzero, see [Control register, DWT_CTRL](#) on page C1-737.
DWT_CTRL.NUMCOMP defines the number of implemented DWT_COMP n registers. Implemented DWT_COMP n registers number from 0 to (NUMCOMP-1). Unimplemented registers are UNK/SBZP.

Attributes See [Table C1-21 on page C1-736](#).

The DWT_COMP n register bit assignments are:



COMP, bits[31:0] Reference value for comparison.

For more information see [The DWT comparators on page C1-719](#).

C1.8.16 Comparator Mask registers, DWT_MASK n

The DWT_MASK n register characteristics are:

Purpose Provides the size of the ignore mask applied to the access address for address range matching by comparator n .

MATCHED, bit[24]	Comparator match: 0 No match. 1 Match. A value of 1 indicates that the operation defined by the FUNCTION field occurred since the last read of the register. Reading the register clears this bit to 0. This bit is read-only.
Bits[23:20]	Reserved.
DATAVADDR1, bits[19:16]	When the DATAVMATCH and LNK1ENA bits are both 1, this field can hold the comparator number of a second comparator to use for linked address comparison. For more information see LinkAddr support on page C1-728. The DWT unit ignores the value of this field unless the LNK1ENA bit is RAO and the DATAVMATCH bit is set to 1. If LNK1ENA is RAZ, this field is RAZ/WI.
DATAVADDR0, bits[15:12]	When the DATAVMATCH bit is set to 1 this field can hold the comparator number of a comparator to use for linked address comparison. For more information see LinkAddr support on page C1-728. The DWT unit ignores the value of this field if the DATAVMATCH bit is set to 0.
DATAVSIZE, bits[11:10]	For data value matching, specifies the size of the required data comparison: 00 Byte. 01 Halfword. 10 Word. The value 0b11 is reserved. Using this value means behavior is UNPREDICTABLE.
LNK1ENA, bit[9]	Indicates whether the implementation supports use of a second linked comparator: 0 Second linked comparator not supported. 1 Second linked comparator supported. When LNK1ENA is RAO, the DATAVADDR1 field specifies the comparator to use as the second linked comparator. This bit is read-only.
DATAVMATCH, bit[8]	Enables data value comparison, if supported: 0 Perform address comparison. 1 Perform data value comparison. For comparator 0, when the CYCMATCH is set to 1, DATAVMATCH must be set to 0 for it to perform cycle count comparison. See LNK1ENA, DATAVSIZE, DATAVADDR0 and DATAVADDR1 for related information. If the implementation does not support data value comparison this bit is RAZ/WI.
CYCMATCH, bit[7]	DWT_FUNCTION0 only. If the implementation supports cycle counting, enable cycle count comparison for comparator 0: 0 No comparison is performed. 1 Compare DWT_COMP0 with the cycle counter, DWT_CYCCNT.

	If DWT_CTRL.NOCYCCNT is RAZ then this bit is UNK/SBZP.
Bits[7]	DWT_FUNCTION n , for all values of n other than 0. Reserved, UNK/SBZP.
Bits[6]	Reserved.
EMITRANGE, bits[5]	If the implementation supports trace sampling, enables generation of Data trace address packets, that hold Daddr[15:0]: 0 Data trace address packets disabled. 1 Enable Data trace address packet generation. For more information see Address comparison functions on page C1-721. If DWT_CTRL.NOTRCPKT is RAZ then this bit is UNK/SBZP.
Bits[4]	Reserved.
FUNCTION, bits[3:0]	Selects action taken on comparator match: 0000 = Disabled or LinkAddr(), see LinkAddr support on page C1-728. For non-zero values: • If DATAVMATCH is set to 1, see Table C1-16 on page C1-726 • If DATAVMATCH is set to 0 then: — If CYCMATCH is set to 0, see Table C1-14 on page C1-722 — If CYCMATCH is set to 1, see Table C1-15 on page C1-724. This field resets to zero. For more information see The DWT comparators on page C1-719.

C1.9 Embedded Trace Macrocell support

An *Embedded Trace Macrocell* (ETM) is an optional feature of an Armv7-M implementation. Where it is implemented, the device must implement a Trace Port Interface Unit that can format the combined output packet stream from:

- The ETM.
- The DWT and ITM.

An ETM implementation must comply with the ETM architecture v3.4 or later, see the appropriate Arm Trace Architecture Specification. The associated TPIU implementation must be CoreSight compliant, see the *Arm® CoreSight™ Architecture Specification* and comply with the TPIU architecture, for compatibility with Arm and other CoreSight-compatible debug solutions.

When an Armv7-M implementation includes an ETM:

- The CMPMATCH[N] signals from the DWT unit are available as control inputs to the ETM unit. For more information see [CMPMATCH\[N\] event generation on page C1-730](#).
- An implementation can use the DEMCR.TRCENA bit as an enable signal for the ETM unit, see [Debug Exception and Monitor Control Register, DEMCR on page C1-706](#).

———— **Note** ————

Whether the TRCENA bit enables the ETM is IMPLEMENTATION DEFINED. This functionality might be inappropriate if the ETM unit is a shared resource in a complex system.

—————